

Lembar Kerja Praktikum Kecerdasan Buatan LKP 3b

Algoritme Pencarian: Uninformed Search

Nama: Mochamad Aleandre Moulidouane
NIM: M0403241105

Kasus 1 – Jalur Distribusi Pupuk di Jawa Tengah

Seorang petani ingin mendistribusikan pupuk dari Gudang Utama (Semarang) ke DesaA. Graf berikut menyatakan jaringan distribusi dengan bobot berupa jarak (km).

Adjacency list:

- Semarang: Kendal(20), Ungaran(15)
- Kendal: Semarang(20), Batang(25)
- Batang: Kendal(25), Pekalongan(30)
- Ungaran: Semarang(15), Ambarawa(10), Salatiga(20)
- Ambarawa: Ungaran(10), Magelang(25)
- Salatiga: Ungaran(20), Boyolali(18)
- Boyolali: Salatiga(18), Solo(22)
- Solo: Boyolali(22), Sragen(25), DesaA(30)
- Magelang: Ambarawa(25), DesaA(40)
- Sragen: Solo(25), DesaA(15)
- DesaA: Magelang(40), Sragen(15), Solo(30)

Kasus 2 – Rute Kapal Pengiriman Hasil Laut di Sulawesi

Nelayan ingin mengirim hasil laut dari Pelabuhan Utama (Makassar) ke PasarB. Graf berikut menyatakan jalur laut antar pelabuhan dengan bobot berupa waktu tempuh (jam).

Adjacency list:

- Makassar: ParePare(5), Jeneponto(4)
- ParePare: Makassar(5), Palopo(7)
- Palopo: ParePare(7), Kolaka(6)
- Jeneponto: Makassar(4), Bira(3)
- Bira: Jeneponto(3), Kendari(8)
- Kolaka: Palopo(6), Kendari(5)
- Kendari: Kolaka(5), Bira(8), BauBau(6)
- BauBau: Kendari(6), PasarB(4)
- PasarB: BauBau(4)

A. Tugas

1. Gambar graf dan tulis ulang *adjacency list*.
2. Lakukan BFS, DFS, dan UCS secara manual dari node awal ke tujuan pada Kasus 1 dan Kasus 2.
 - Tampilkan urutan ekspansi node.
 - Tampilkan isi frontier/stack/priority queue pada tiap langkah.
 - Tuliskan solusi jalur yang ditemukan serta biaya total (untuk UCS).

i. BFS

Iterasi	Node diekspansi	Frontier (Queue) setelah ekspansi	Explored	Solusi ditemukan?

ii. DFS

Iterasi	Node diekspansi	Frontier (Stack) setelah ekspansi	Explored	Solusi ditemukan?

iii. UCS

Iterasi	Node diambil (biaya)	Frontier (PriorityQueue) setelah ekspansi	Explored	Solusi ditemukan?

3. Bandingkan hasil ketiga algoritme.

4. Implementasikan dengan AIMA Python dan bandingkan dengan hasil manual.

5. Jawablah pertanyaan reflektif.

B. Jawab

Kasus 1 – Jalur Distribusi Pupuk di Jawa Tengah

1. Adjacency list:

Semarang: Kendal(20), Ungaran(15)

Kendal: Semarang(20), Batang(25)

Batang: Kendal(25), Pekalongan(30)

Ungaran: Semarang(15), Ambarawa(10), Salatiga(20)

Ambarawa: Ungaran(10), Magelang(25)

Salatiga: Ungaran(20), Boyolali(18)

Boyolali: Salatiga(18), Solo(22)

Solo: Boyolali(22), Sragen(25), DesaA(30)

Magelang: Ambarawa(25), DesaA(40)

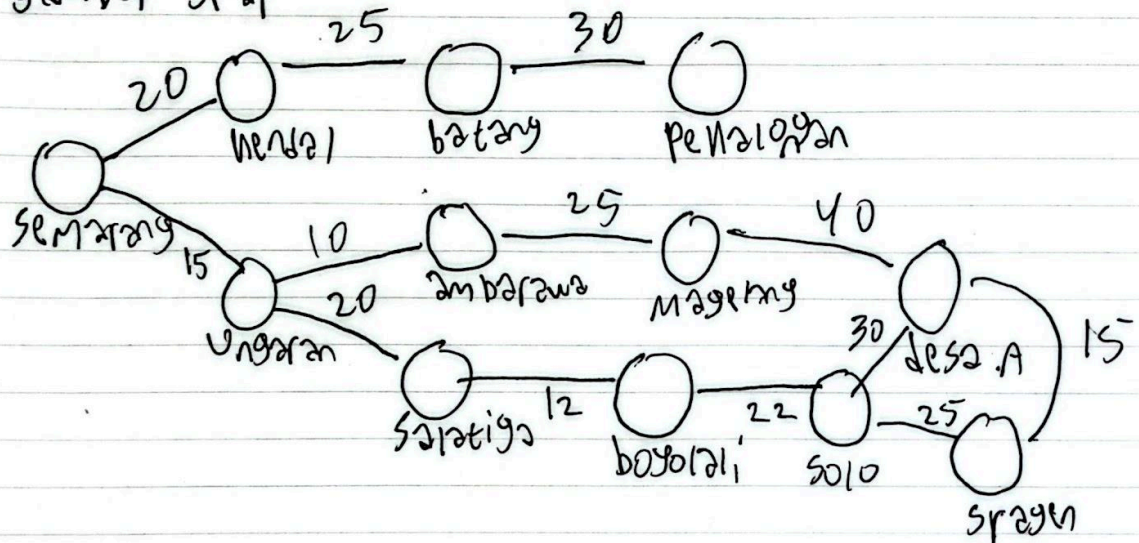
Sragen: Solo(25), DesaA(15)

DesaA: Magelang(40), Sragen(15), Solo(30)

Titik awal Semarang menuju titik tujuan DesaA

2. Gambar Graf

LKP 3b
 Versus 1
 gambar graf



3. BFS

Iterasi	Node diekspansi	Frontier (Queue) setelah ekspansi	Explored	Solusi?
0	Semarang	Kendal, Ungaran	Semarang	Tidak
1	Kendal	Ungaran, Batang	Semarang, Kendal	Tidak
2	Ungaran	Batang, Ambarawa, Salatiga	Semarang, Kendal, Ungaran	Tidak
3	Batang	Ambarawa, Salatiga, Pekalongan	..., Batang	Tidak
4	Ambarawa	Salatiga, Pekalongan, Magelang	..., Ambarawa	Tidak
5	Salatiga	Pekalongan, Magelang, Boyolali	..., Salatiga	Tidak
6	Pekalongan	Magelang, Boyolali	..., Pekalongan	Tidak
7	Magelang	Boyolali, DesaA	..., Magelang	YA

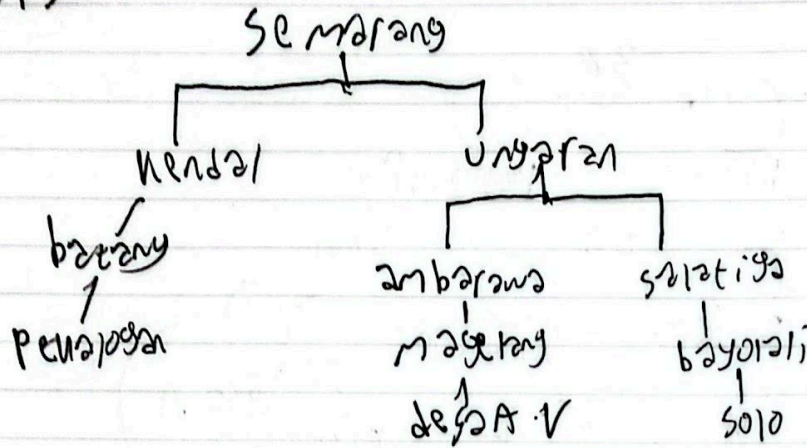
Jalur:

Semarang -> Ungaran -> Ambarawa -> Magelang -> DesaA

Total cost: $15 + 10 + 25 + 40 = 90$ km.

Struktur pohon penelusuran:

TRE BFS :



4. DFS

Iterasi	Node diekspansi	Isi stack setelah ekspansi	Explored	Solusi ?
0	Semarang	Ungaran, Kendal	Semarang	Tidak
1	Ungaran	Salatiga, Ambarawa, Kendal	Semarang, Ungaran	Tidak
2	Salatiga	Boyolali, Ambarawa, Kendal	..., Salatiga	Tidak
3	Boyolali	Solo, Ambarawa, Kendal	..., Boyolali	Tidak
4	Solo	DesaA, Sragen, Ambarawa, Kendal	..., Solo	Tidak
5	DesaA	-	..., DesaA	YA

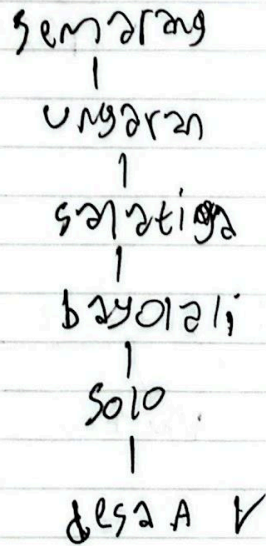
Jalur:

Semarang -> Ungaran -> Salatiga -> Boyolali -> Solo -> DesaA

Total cost: $15 + 20 + 18 + 22 + 30 = 105$ km.

Struktur pohon penelusuran DFS:

Tree DFS:



5. UCS

Iterasi	Node diambil (biaya)	Frontier (PriorityQueue) setelah ekspansi	Explored	Solusi ?
0	Semarang (0)	Ungaran:15, Kendal:20	Semarang	Tidak
1	Ungaran (15)	Kendal:20, Ambarawa:25, Salatiga:35	..., Ungaran	Tidak
2	Kendal (20)	Ambarawa:25, Salatiga:35, Batang:45	..., Kendal	Tidak
3	Ambarawa (25)	Salatiga:35, Batang:45, Magelang:50	..., Ambarawa	Tidak
4	Salatiga (35)	Batang:45, Magelang:50, Boyolali:53	..., Salatiga	Tidak
5	Batang (45)	Magelang:50, Boyolali:53, Pekalongan:75	..., Batang	Tidak
6	Magelang (50)	Boyolali:53, Pekalongan:75, DesaA:90	..., Magelang	Tidak
7	DesaA (90)	-	..., DesaA	YA

Solusi jalur UCS:

Semarang -> Ungaran -> Ambarawa -> Magelang -> DesaA

Total cost paling optimal: 90 km.

6. Perbandingan kode

```

import os
import sys

sys.path.append(os.path.abspath('../aima-python'))

from aima.search import Graph, GraphProblem, breadth_first_graph_search,
depth_first_graph_search, uniform_cost_search
  
```

```

jalurDistribusi = Graph(dict(
    Semarang=dict(Kendal=20, Ungaran=15),
    Kendal=dict(Semarang=20, Batang=25),
    Batang=dict(Kendal=25, Pekalongan=30),
    Ungaran=dict(Semarang=15, Ambarawa=10, Salatiga=20),
    Ambarawa=dict(Ungaran=10, Magelang=25),
    Salatiga=dict(Ungaran=20, Boyolali=18),
    Boyolali=dict(Salatiga=18, Solo=22),
    Solo=dict(Boyolali=22, Sragen=25, DesaA=30),
    Magelang=dict(Ambarawa=25, DesaA=40),
    Sragen=dict(Solo=25, DesaA=15),
    DesaA=dict(Magelang=40, Sragen=15, Solo=30)
), directed=False)

problem = GraphProblem('Semarang', 'DesaA', jalurDistribusi)

print("Kasus 1 - Jalur Distribusi Pupuk di Jawa Tengah")

print("1. BFS Path:", breadth_first_graph_search(problem).path(), "\nCost:",
breadth_first_graph_search(problem).path_cost, "KM")
print("2. DFS Path:", depth_first_graph_search(problem).path(), "\nCost:",
depth_first_graph_search(problem).path_cost, "KM")
print("3. UCS Path:", uniform_cost_search(problem).path(), "\nCost:",
uniform_cost_search(problem).path_cost, "KM")

```

Hasil:

```

Kasus 1 - Jalur Distribusi Pupuk di Jawa Tengah
1. BFS Path: [<Node Semarang>, <Node Ungaran>, <Node Ambarawa>, <Node Magelang>, <Node DesaA>]
Cost: 90 KM
2. DFS Path: [<Node Semarang>, <Node Ungaran>, <Node Salatiga>, <Node Boyolali>, <Node Solo>, <Node DesaA>]
Cost: 105 KM
3. UCS Path: [<Node Semarang>, <Node Ungaran>, <Node Ambarawa>, <Node Magelang>, <Node DesaA>]
Cost: 90 KM

```

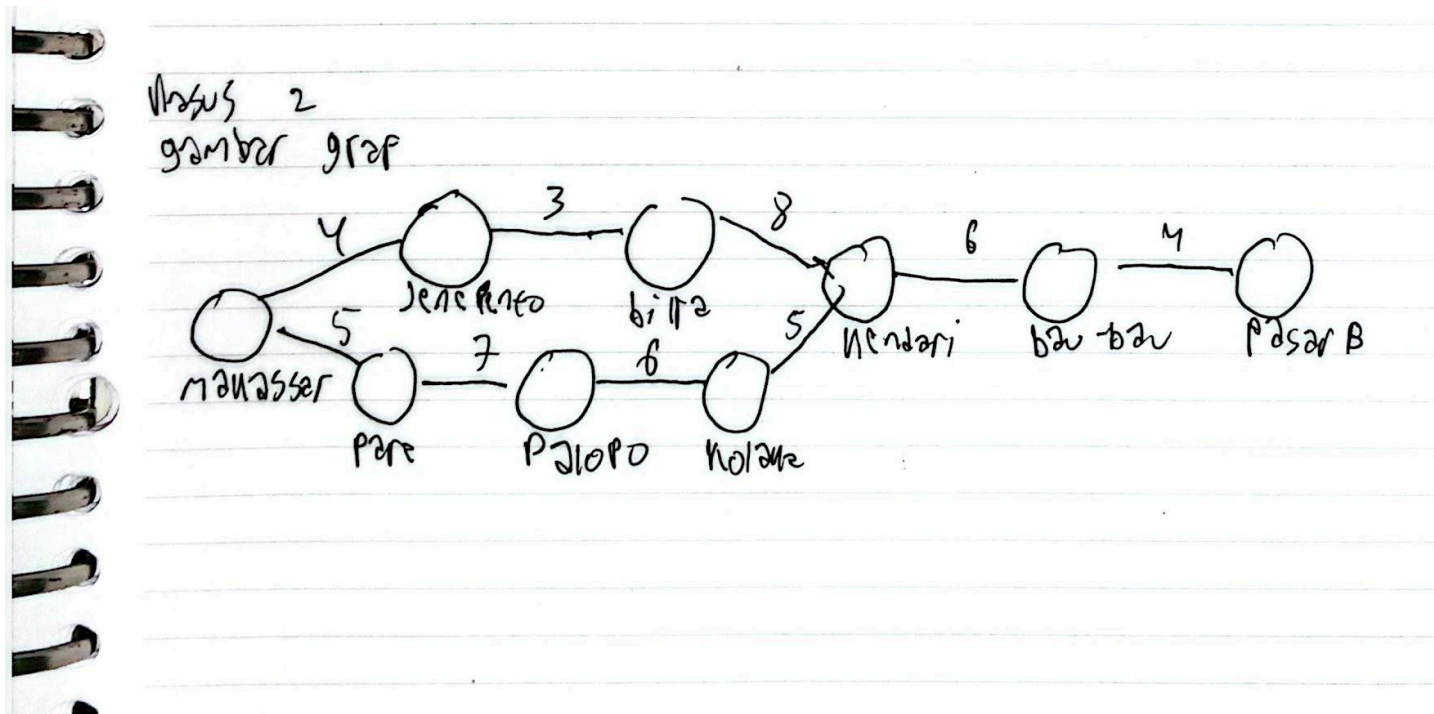
Kasus 2 – Rute Kapal di Sulawesi Selatan

1. Adjacency list:

Makassar: ParePare(5), Jeneponto(4)
 ParePare: Makassar(5), Palopo(7)
 Palopo: ParePare(7), Kolaka(6)
 Jeneponto: Makassar(4), Bira(3)
 Bira: Jeneponto(3), Kendari(8)
 Kolaka: Palopo(6), Kendari(5)
 Kendari: Kolaka(5), Bira(8), BauBau(6)
 BauBau: Kendari(6), PasarB(4)
 PasarB: BauBau(4)

Titik asal Makassar titik akhir PasarB.

2. Gambar Graf



3. BFS

Iterasi	Node diekspansi	Frontier setelah ekspansi	Explored	Solusi ditemukan?
0	Makassar	ParePare, Jeneponto	Makassar	Tidak
1	ParePare	Jeneponto, Palopo	Makassar, ParePare	Tidak
2	Jeneponto	Palopo, Bira	..., Jeneponto	Tidak
3	Palopo	Bira, Kolaka	..., Palopo	Tidak
4	Bira	Kolaka, Kendari	..., Bira	Tidak
5	Kolaka	Kendari	..., Kolaka	Tidak
6	Kendari	BauBau	..., Kendari	Tidak
7	BauBau	PasarB	..., BauBau	Tidak
8	PasarB	-	..., PasarB	YA

Jalur:

Makassar -> Jeneponto -> Bira -> Kendari -> BauBau -> PasarB

Total cost: $4 + 3 + 8 + 6 + 4 = 25$ jam.

4. DFS

Iterasi	Node diekspansi	Isi stack setelah ekspansi	Explored	Solusi ditemukan?
0	Makassar	Jeneponto, ParePare	Makassar	Tidak
1	Jeneponto	Bira, ParePare	Makassar, Jeneponto	Tidak
2	Bira	Kendari, ParePare	..., Bira	Tidak

3	Kendari	BauBau, Kolaka, ParePare	..., Kendari	Tidak
4	BauBau	PasarB, Kolaka, ParePare	..., BauBau	Tidak
5	PasarB	-	..., PasarB	YA

Jalur:

Makassar -> Jenepono -> Bira -> Kendari -> BauBau -> PasarB

Total cost: $4 + 3 + 8 + 6 + 4 = 25$ jam.

5. UCS

Iterasi	Node diekspansi	Priority queue setelah ekspansi	Explored	Solusi ditemukan?
0	Makassar (0)	Jenepono:4, ParePare:5	Makassar	Tidak
1	Jenepono (4)	ParePare:5, Bira:7	..., Jenepono	Tidak
2	ParePare (5)	Bira:7, Palopo:12	..., ParePare	Tidak
3	Bira (7)	Palopo:12, Kendari:15	..., Bira	Tidak
4	Palopo (12)	Kendari:15, Kolaka:18	..., Palopo	Tidak
5	Kendari (15)	Kolaka:18, BauBau:21	..., Kendari	Tidak
6	Kolaka (18)	BauBau:21	..., Kolaka	Tidak
7	BauBau (21)	PasarB:25	..., BauBau	Tidak
8	PasarB (25)	-	..., PasarB	YA

Jalur UCS:

Makassar -> Jenepono -> Bira -> Kendari -> BauBau -> PasarB

Total cost paling optimal: 25 jam.

6. Perbandingan Code

```
RuteKapal = Graph(dict(
    Makassar=dict(ParePare=5, Jenepono=4),
    ParePare=dict(Makassar=5, Palopo=7),
    Palopo=dict(ParePare=7, Kolaka=6),
    Jenepono=dict(Makassar=4, Bira=3),
    Bira=dict(Jenepono=3, Kendari=8),
    Kolaka=dict(Palopo=6, Kendari=5),
    Kendari=dict(Kolaka=5, Bira=8, BauBau=6),
    BauBau=dict(Kendari=6, PasarB=4),
    PasarB=dict(BauBau=4)
), directed=False)

problem2 = GraphProblem('Makassar', 'PasarB', RuteKapal)

print("\n", "Kasus 2 - Rute Kapal di Sulawesi Selatan")
```

```
print("1. BFS Path:", breadth_first_graph_search(problem2).path(),
"\nCost:", breadth_first_graph_search(problem2).path_cost, "Jam")
print("2. DFS Path:", depth_first_graph_search(problem2).path(),
"\nCost:", depth_first_graph_search(problem2).path_cost, "Jam")
print("3. UCS Path:", uniform_cost_search(problem2).path(),
"\nCost:", uniform_cost_search(problem2).path_cost, "Jam")
```

Hasil:

Kasus 2 - Rute Kapal di Sulawesi Selatan

```
1. BFS Path: [<Node Makassar>, <Node Jeneponto>, <Node Bira>, <Node Kendari>, <Node BauBau>, <Node PasarB
Cost: 25 Jam
2. DFS Path: [<Node Makassar>, <Node Jeneponto>, <Node Bira>, <Node Kendari>, <Node BauBau>, <Node PasarB
Cost: 25 Jam
3. UCS Path: [<Node Makassar>, <Node Jeneponto>, <Node Bira>, <Node Kendari>, <Node BauBau>, <Node PasarB
Cost: 25 Jam
```

Selengkapnya bisa cek di:

<https://github.com/VSaleXD/kecerdasanBuatan/blob/main/hari3/LKP3b.md>

C. Pertanyaan Reflektif

1. Mengapa BFS dan UCS bisa menghasilkan solusi yang berbeda pada graf berbobot?

Jawab :

BFS dan UCS bisa menghasilkan solusi yang berbeda pada graf berbobot dikarenakan BFS dapat diasumsikan sebagai algoritma pencarian tanpa mempedulikan bobot / semua edge dianggap memiliki bobot yang sama, sedangkan UCS memperhitungkan bobot yang diperlukan oleh setiap edge untuk algoritma pencariannya sehingga UCS akan menghasilkan solusi dengan memperhitungkan bobot yang dimiliki setiap edge. Sehingga BFS dan UCS dapat menghasilkan solusi yang berbeda pada graf berbobot dikarenakan BFS tidak mempedulikan bobot edge sedangkan UCS mempertimbangkan bobot setiap edge.

2. Pada kondisi apa DFS dapat menemukan solusi yang sama dengan BFS?

Jawab :

DFS akan menemukan solusi yang sama dengan BFS saat graf yang digunakan untuk algoritma pencarian memiliki jarak yang sangat pendek atau grafnya hanya 1 arah. BFS dan DFS dapat memiliki solusi yang sama juga pada kasus dimana solusi dari graf tersebut berada di level / di jalur kiri / jalur paling awal graf.

3. Jika semua bobot = 1, jelaskan relasi hasil BFS dan UCS.

Jawab :

Jika semua bobot graf adalah 1 maka relasi hasil BFS dan UCS adalah akan sama persis. Karena jika bobot graf sama persis bernilai 1 maka tidak ada perhitungan bobot dikarenakan semua bobot dianggap sama sehingga apabila semua bobot bernilai 1 maka UCS akan bernilai sama dengan BFS, karena basis dari UCS sendiri yakni BFS dengan priority queue.

4. Bagaimana urutan tetangga mempengaruhi hasil DFS?

Jawab :

Urutan tetangga dapat mempengaruhi hasil DFS dikarenakan jika tetangga diurutkan berbeda maka hasil DFS juga akan berbeda tergantung tetangga mana yang ditelusuri lebih dulu, hal ini menyebabkan DFS dapat bekerja lebih cepat ataupun lebih lambat. Pada graf yang sama pun DFS bisa menghasilkan hasil yang berbeda beda tergantung tetangga mana yang dimasukkan ke stack dahulu.